

PART 1 – WHAT IS PYTHON IN DATA ENGINEERING?

Definition

Python is a high-level, interpreted, dynamically typed programming language used to build data pipelines, automation scripts, transformation logic, APIs, and distributed data applications.

In Data Engineering, Python is used to:

- Ingest data from files and APIs
- Clean and transform datasets
- Automate workflows
- Write Spark jobs
- Build Airflow DAGs
- Integrate with databases

Before Spark.

Before Pandas.

Before distributed systems.

Everything begins with:

- Variables
- Data structures
- Functions
- Error handling
- File processing

PART 2 – PYTHON EXECUTION MODEL

Definition: Variable

A variable is a reference to an object stored in memory.

Example:

```
a = 10
```

This does NOT store “10 inside a”.

It:

1. Creates an integer object in memory.
2. Assigns reference of that object to variable a.

Object Identity

Every object in Python has:

- Type
- Value
- Identity (memory address)

Example:

```
a = 10
```

```
b = 10
```

Mutability – Formal Definition

An object is mutable if its internal state can be changed after creation.

An object is immutable if it cannot be altered after creation.

Immutable Types

- int
- float
- str
- tuple
- bool

Example:

```
name = "John"  
name.upper()
```

name does not change. A new string is created.

Mutable Types

- list
- dict
- set

Example:

```
data = [1,2,3]  
data.append(4)
```

The same object changes.

Engineering Implication

If multiple variables reference same mutable object:

```
a = [1,2,3]
b = a
b.append(4)
```

Now both change.

This causes hidden side effects in pipelines.

Correct defensive approach:

```
b = a.copy()
```

Understanding this prevents:

- Silent corruption
- Hard-to-trace bugs
- Unexpected transformation behavior

PART 3 – DATA STRUCTURES

LIST

Definition

A list is an ordered, mutable collection that can store elements of different types.

Internal Behavior

Python lists are implemented as dynamic arrays.

They:

- Maintain order
- Allow indexing
- Automatically resize

Core Properties

Property	Value
Ordered	Yes
Mutable	Yes
Allows duplicates	Yes
Indexed	Yes

Example

```
rows = ["John,25,5000", "Alice,30,7000"]
```

Indexing

```
rows[0]  
rows[-1]
```

Negative indexing counts from end.

Modifying

```
rows.append("Bob,40,9000")  
rows.insert(0, "Header")  
rows.remove("Alice,30,7000")
```

Data Engineering Use Cases

- Temporary storage of rows
- Buffering data
- Collecting transformed results
- Passing batch to API

Exercise 1

Given:

```
data = ["100", "200", "abc", "300", ""]
```

Task:

- Convert valid numeric values to integers.
- Skip invalid entries.
- Output clean list.

TUPLE

Definition

A tuple is an ordered, immutable sequence.

Properties

Property	Value
Ordered	Yes
Mutable	No

Allows duplicates	Yes
Indexed	Yes

Example

```
record = ("John", 25, 5000)
```

Unpacking

```
name, age, salary = record
```

Why Tuples Matter

Used for:

- Fixed structure rows
- Database result sets
- Composite keys
- Configuration constants

Immutability ensures integrity.

Exercise 2

Create tuple representing:

```
(order_id, customer_id, total_amount)
```

Unpack and print formatted statement.

SET

Definition

A set is an unordered collection of unique elements.

Internally implemented using a hash table.

Properties

Property	Value
Ordered	No
Mutable	Yes
Allows duplicates	No
Indexed	No

Example

```
ids = {101, 102, 103, 101}
```

Output:

```
{101,102,103}
```

Membership Testing

```
101 in ids
```

Data Engineering Use Case

Deduplication before warehouse insert.

Example:

```
unique_ids = set(incoming_ids)
```

Exercise 3

Given list of transaction IDs:

Remove duplicates.

Check if ID 500 exists.

Add if missing.

DICTIONARY

Definition

A dictionary is a mutable mapping of unique keys to values.

Internally implemented as a hash table.

Properties

Property	Value
Ordered (Python 3.7+)	Yes
Mutable	Yes
Keys unique	Yes
Indexed by key	Yes

Example

```
row = {  
    "name": "John",  
    "age": "25",  
    "salary": "5000"  
}
```

Access Methods

```
row["name"]  
row.get("age")
```

Difference:

- `[]` raises `KeyError`
- `.get()` returns `None` if missing

Update

```
row["age"] = int(row["age"])
```

Schema enforcement step.

Nested Dictionary

```
response = {  
    "user": {  
        "profile": {  
            "name": "Alice"  
        }  
    }  
}
```

```
}  
}
```

Safe access:

```
response.get("user", {}).get("profile", {}).get("name")
```

Prevents crashes.

Exercise 4

Given:

```
row = {"name": "Tom", "age": "", "salary": "4000"}
```

Tasks:

- Convert salary to float.
- Replace empty age with None.
- Add key "processed": True.

PART 4 – FUNCTIONS

Definition

A function is a reusable block of code that performs a specific task and optionally returns a value.

Basic Structure

```
def function_name(parameters):  
    logic  
    return result
```

Why Functions Matter in Data Engineering

- Modular design
- Reusability
- Easier testing
- Cleaner architecture
- Maintainable pipelines

Example – Clean Row

```
def clean_row(row):  
    name = row["name"].strip()  
    age = int(row["age"])  
    salary = float(row["salary"]) if row["salary"] else 0  
  
    return {  
        "name": name,  
        "age": age,  
        "salary": salary  
    }
```

Design Principles

- One responsibility
- No global variable modification
- Always return data
- Avoid print inside logic

Exercise 5

Write function:

```
validate_row(row)
```

Rules:

- Age must be integer and ≥ 0
- Salary must be ≥ 0
- Return True if valid else False

PART 5 – EXCEPTION HANDLING

Definition

An exception is an error event that disrupts normal program flow.

Basic Example

```
int("abc")
```

Raises ValueError.

Try/Except Structure

```
try:  
    value = int(data)
```

```
except ValueError:  
    print("Invalid value")
```

Raising Exceptions

```
if age < 0:  
    raise ValueError("Age cannot be negative")
```

Used for enforcing business rules.

Exercise 6

Modify `clean_row()` to:

- Raise error if salary negative
- Catch exception and log error

PART 6 – LOGGING

Definition

Logging is structured recording of program events for monitoring and debugging.

Setup

```
import logging  
  
logging.basicConfig(
```

```
    filename="pipeline.log",  
    level=logging.INFO,  
    format="%(asctime)s - %(levelname)s - %(message)s"  
)
```

Levels

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

Logging Example

```
logging.info("Pipeline started")  
logging.error("Invalid row encountered")
```

Exercise 7

Add counters:

- total_rows
- valid_rows
- invalid_rows

Log summary at end.

PART 7 – FILE I/O

Reading File

```
with open("data.csv", "r") as file:
    for line in file:
        print(line.strip())
```

Writing File

```
with open("output.csv", "w") as file:
    file.write("John,25,5000\n")
```

CSV Module

```
import csv

with open("data.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)
```

Handles:

- Headers
- Quoted fields
- Commas in values

FINAL LAB – COMPLETE CSV PIPELINE

Input:

```
name,age,salary
John,25,5000
Alice,,7000
```

Bob,abc,4000

Tom,30,

Requirements:

- Convert types
- Replace missing salary with 0
- Skip invalid age
- Log invalid rows
- Maintain metrics
- Write cleaned output

Full implementation provided earlier.

ADVANCED CHALLENGE

Enhance pipeline to:

- Stop job if invalid rows > 30%
- Accept input filename as argument
- Add execution time measurement
- Add DEBUG mode switch